

Q1. Uniform Cost Search (UCS)

Problem

A logistics company wants to transport packages from **Warehouse S (Start)** to **Warehouse G (Goal)**.

Each road has a **cost**:

- Fuel cost
- Time cost

The company wants the **least-cost path**.

Why Uniform Cost Search?

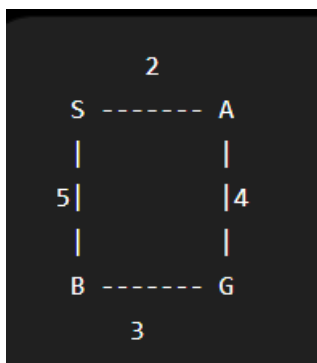
Uniform Cost Search always chooses the path with the **smallest total cost**.

It does **not** care about the fewest number of roads.

It only cares about the **minimum total cost**.

Example

Suppose the graph is:



Possible paths:

- **S → A → G**
 - Cost = 2 + 4 = **6**
- **S → B → G**
 - Cost = 5 + 3 = **8**

UCS selects:

S → A → G

because $6 < 8$.

How UCS Works

1. Start from **S**.
2. Put **S** into a priority queue with cost **0**.
3. Remove the node with the **lowest total cost**.
4. Expand its neighbors.
5. Update their costs.
6. Repeat until **G** is reached.
7. The first time **G** is removed from the queue, that path is the least-cost path.

Pseudocode

UCS(Start, Goal)

Create Priority Queue

Insert Start with cost = 0

WHILE Queue is not empty

 Remove node with minimum cost

 IF node == Goal

 Return Path and Cost

 FOR each neighbor

 NewCost = CurrentCost + EdgeCost

 IF neighbor not visited

Insert neighbor with NewCost

END WHILE

Exam Answer (5 Marks)

Algorithm: Uniform Cost Search (UCS)

Reason: UCS always expands the node with the minimum cumulative path cost and guarantees the least-cost path.

Steps:

1. Start from source node **S**.
2. Insert **S** into a priority queue with cost **0**.
3. Expand the node with the smallest cost.
4. Add neighboring nodes with updated costs.
5. Repeat until destination **G** is reached.
6. Return the least-cost path.

Q2. Mars Rover – AI Agent

i. What are the percepts for this agent?

Percepts are the information the Mars Rover receives from its sensors.

Examples:

- Images of rocks and soil
- Surface temperature
- Obstacle detection
- Distance to objects
- Atmospheric conditions
- Battery level
- Position (GPS-like localization on Mars)

Answer:

The percepts are camera images, soil and rock data, temperature, obstacles, atmospheric information, battery status, and location.

ii. Characterize the operating environment.

The operating environment describes where the Mars Rover works.

Characteristics:

- **Partially observable** – The rover cannot see everything around it.
- **Deterministic/Stochastic** – Mostly uncertain because dust storms and terrain can change.
- **Sequential** – Each action affects future actions.
- **Dynamic** – The environment changes over time.
- **Continuous** – Movement and sensing happen continuously.
- **Single-agent** – Only one rover performs the task.

Answer:

The Mars Rover operates in a partially observable, uncertain, sequential, dynamic, continuous, and single-agent environment.

iii. What actions can the agent take?

The rover can:

- Move forward, backward, left, and right
- Avoid obstacles
- Collect rock and soil samples
- Drill rocks
- Capture photos and videos
- Analyze samples
- Send collected data to Earth
- Recharge using solar panels (if available)

Answer:

The rover can move, avoid obstacles, collect samples, analyze rocks, capture images, communicate with Earth, and manage its power.

iv. How can one evaluate the performance of the agent?

The rover's performance is measured by:

- Successfully collecting samples
- Accuracy of scientific analysis
- Safe navigation
- Number of obstacles avoided
- Battery efficiency
- Amount of useful data sent to Earth
- Completing the mission within the available time

Answer:

The agent is evaluated based on mission success, safe navigation, sample collection, scientific accuracy, energy efficiency, and successful communication.

v. What sort of agent architecture is most suitable? Why?

Suitable Architecture: Utility-Based Learning Agent

Why?

- Learns from previous experiences.

- Chooses the best action in uncertain situations.
- Balances multiple goals like safety, energy saving, and scientific exploration.
- Adapts to unknown environments on Mars.

Answer:

A **Utility-Based Learning Agent** is the most suitable because it can learn from experience, make intelligent decisions, adapt to changing conditions, and maximize mission success.

Q3. OLA Cab Booking Problem

Question

A customer wants to travel from one location to another using the OLA Cab app. The customer can choose a cab type (Mini, Micro, Sedan, Shared, Prime, etc.) based on comfort.

Identify the type of problem-solving agent and write the pseudocode.

1. Which type of problem-solving agent is used?

Answer: Goal-Based Problem-Solving Agent

Why?

The customer's **goal** is to **reach the destination**.

The agent:

- Takes the source and destination.
- Finds available cab types.
- Allows the user to choose a cab.
- Books the selected cab.
- Helps the customer reach the destination.

Since it works toward achieving a specific goal, it is called a **Goal-Based Agent**.

2. State Space

In AI, a problem-solving agent is explained using four components.

- **Initial State:** Customer opens the OLA app.
- **Actions:** Enter pickup & destination, choose cab type, book cab.
- **Goal State:** Customer reaches the destination.
- **Path Cost:** Fare, travel time, or distance.

3. Pseudocode

START

Input Pickup_Location

Input Destination

Display available cab types

(Mini, Micro, Sedan, Shared, Prime)

User selects cab type

IF cab is available THEN

 Show fare and estimated arrival time

 Confirm booking

 Assign nearest driver

 Driver picks up customer

 Customer reaches destination

 Display "Trip Completed"

ELSE

 Display "No Cab Available"

ENDIF

STOP

4. Simple Explanation

1. Customer opens the OLA app.
2. Enters pickup and destination.
3. App searches for nearby cabs.
4. Customer selects a cab type (Mini, Sedan, Prime, etc.).
5. If a cab is available, the booking is confirmed.
6. The driver arrives and takes the customer to the destination.

5. Exam Answer (5 Marks)

Type of Agent: Goal-Based Problem-Solving Agent

Reason: It aims to achieve the customer's goal of reaching the destination by selecting and booking the most suitable cab.

Pseudocode:

START

Input source and destination

Search available cabs

Display cab types

User selects cab

IF cab available

 Book cab

 Assign driver

 Travel to destination

 Trip completed

ELSE

 Show "No cab available"

END IF

STOP